



Programa de Iniciación Tecnológica

Git y GitHub

Control de Versiones Software

Sesión 3 • Ramas, fusiones y conflictos

Luis Rojas

Instructor del curso • Universidad Nacional de Ingeniería

Modalidad virtual (Zoom + YouTube) • 3 horas con receso

En el aula

- Pregunten sin miedo.
- Equivocarse es parte del oficio.
- Practicamos en vivo, comando a comando.
- Hay receso a mitad de clase (15 a 20 min).

Temas de esta sesión

- Qué es una rama (un puntero, no una carpeta extra).
- Crear, cambiar y listar ramas.
- Fusionar líneas de tiempo: **git merge**.
- Resolver un conflicto con calma.
- Un repo nuevo: una receta de limonada.

De dónde venimos (sesiones 1 y 2)

Si las clases anteriores quedaron hechas, hoy partimos de esto:

- Git $\neq$ GitHub. Hoy seguimos **en local**. No hay **push**.
- Identidad configurada. Un commit es una copia instantánea: hash, autor, mensaje.
- Ya vieron **git log --oneline --graph --all** en **una** línea recta.

Hoy no abrimos el repo de ayer

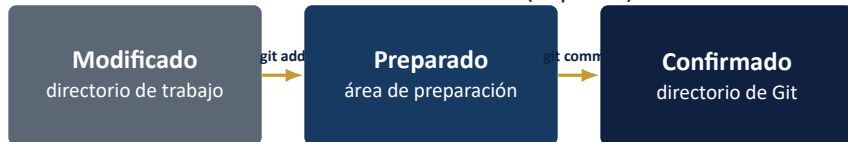
No usamos `mi-primer-repo` ni `practica-historial`. Quien faltó entra por la misma puerta: carpeta nueva, **git init**.

Si Git no arranca

git --version. Si falla, reinstale y abra una terminal **nueva**. En Windows: Git Bash.

Los tres estados, como los nombra Pro Git

En las sesiones 1 y 2 les decíamos *taller*, *bandeja* e *historial*. A partir de hoy usamos los nombres del libro **Pro Git** (cap. 1.3).



- **Directorio de trabajo:** los archivos que ve y edita (antes: taller).
- **Área de preparación** (*staging* / índice): lo marcado para la próxima foto (antes: bandeja).
- **Directorio de Git** (la carpeta `.git`): la base de datos local, ya confirmada (antes: historial / caja fuerte).

Idea clave. La brújula sigue siendo **git status**: dice en cuál de los tres estados está cada archivo.

Al cerrar la sesión 3 deberían poder, con las manos:

- ❶ Explicar una rama con una analogía cotidiana (no con jerga).
- ❷ Crear una rama, trabajar ahí y volver a main sin perder fotos.
- ❸ Fusionar una rama cuando no hay choque.
- ❹ Reconocer los marcadores de un conflicto y dejar **un** archivo coherente.
- ❺ Tener el repo receta-Timonada con main y al menos dos ramas en el dibujo del log.

Idea clave. La meta no es «saber merge». Es no tener miedo cuando el historial deja de ser una línea recta.

La receta que ya funciona

Analogía de hoy: una **receta de limonada**. Todo el mundo entiende una receta; por eso se usa mucho para enseñar ramas.

La receta oficial	Una variación	Adoptar la variación
Lo que ya salió bien. Eso, en Git, es <code>main</code> : no se improvisa encima.	«¿Y si le ponemos menta?» Se prueba en otra línea de tiempo. Si sale mal, <code>main</code> no se enteró.	Cuando convence, se fusiona a la receta oficial. Eso es merge .

Idea clave. Una rama no es otra cocina. Es otra línea de fotos de *la misma* receta.

Qué es una rama (sin misterio)

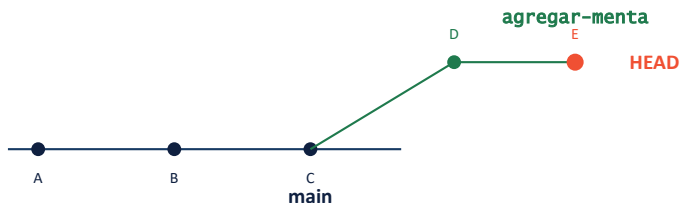
Una rama es un **puntero** con nombre: apunta a un commit. `main` es una rama.



- Hasta ahora ustedes vivían en `main`: una sola línea, HEAD al final.
- Crear una rama **no copia** la carpeta. Solo pone un letrero nuevo en el mismo commit.
- El trabajo nuevo nace cuando hacen commits *estando* en esa rama.

Cuando el historial se parte

Trabajan en una variación (agregar-menta) mientras main se queda en la receta oficial:



- C y E son fotos distintas. El archivo en disco es el de **la rama activa**.
- Si vuelven a `main`, Git reescribe el **directorio de trabajo** para que coincida con C.
- Nadie «perdió» E: sigue en el historial, en la otra rama.

HEAD, otra vez, ahora con ramas

HEAD sigue significando «usted está aquí». La diferencia: ahora puede apuntar a `main` o a otro letrero.

En `main`

La receta oficial. Los commits nuevos alargan esa línea.

En agregar-menta

La variación. Los commits nuevos no tocan lo que ya funciona en `main`.

- **git status** dice la rama activa en la primera línea.
- **git log --oneline --graph --all** dibuja **todas** las líneas, no solo en la que están.
- Cambiar de rama con el **directorio de trabajo** sucio (modificado, sin confirmar) a veces se niega. Hoy: preparen y confirmen, o descarten, *antes* de saltar.

Listar y crear: `git branch` y `git switch`

```
git branch
git switch -c agregar-menta
git branch
git status
```

- **`git branch`**: lista letreros. El asterisco marca dónde está HEAD.
- **`git switch -c nombre`**: crea la rama y se pasa a ella.
- El `-c` es de *create*. Sin él, **`git switch main`** solo cambia de letrero.

Ojo. Escriba los dos guiones y el `-c` con la tecla de menos, al lado del cero. No copie guiones raros desde un PDF o un Word.

```
git switch main  
git status  
git log --oneline --graph --all
```

- **git switch main**: HEAD apunta otra vez a la receta oficial.
- El archivo en disco puede **cambiar de contenido**. No es un bug: Git muestra la foto de esa rama.
- **--all** es lo que hace visible la variación aunque estén en main.

Tutoriales viejos

Verán **git checkout -b** y **git checkout main**. Hoy usamos **switch**: hace lo mismo, con un nombre más claro.

Nombres de rama que se entienden

Una rama es un letrero. Si el letrero miente, el historial no sirve.

Bien

- agregar-menta
- cambia-porciones
- menos-azucar

Mal

- test, asdf, nueva, rama1
- final-final-v2
- Espacios o tildes en el nombre

Minúsculas, guion medio, una intención. Si no pueden decirla en una frase, el nombre no está listo.

Hoy una receta nueva, no el repo de ayer

Todo el ejemplo de hoy vive en **una** carpeta. Los archivos se crean en **Git Bash**, sin editor.

Para qué

- Ver ramas sobre un archivo que se entiende: una receta.
- Fusionar sin choque y después *con* choque.
- Que quien llegó hoy no dependa de la sesión 1 ni de la 2.

Qué van a crear

- Carpeta: receta-limonada.
- Archivos: README.md y receta.md.
- Ramas: main, agregar-menta y más adelante menos-azucar.

Idea clave. Un solo proyecto, de arriba abajo. No mezclen esta carpeta con practica-historial.

Cómo pegar un bloque cat en Git Bash

Hoy **no** hace falta el editor. Copie el bloque entero, desde cat hasta la línea EOF.

- ① Seleccione el recuadro oscuro de la diapositiva (solo el código).
- ② En Git Bash: **clic derecho** para pegar (o Mayús+Insert).
- ③ Pulse Enter. El prompt \$ debe volver a aparecer.
- ④ Compruebe el archivo con **cat README.md** (o el nombre que corresponda).

Tres reglas al copiar

Copie también la línea EOF, sola. No copie el pie de página ni el título de la diapositiva. Los dos guiones de las banderas Git (`--no-edit`) deben ser la tecla de menos, no un raya de Word.

Paso 0: carpeta e `init`

Pegue esto en Git Bash:

```
mkdir -p ~/curso-git/sesion-03
cd ~/curso-git/sesion-03
mkdir receta-limonada
cd receta-limonada
git init
git status
```

La primera línea de status debe decir `On branch main`. Si dice `master`:

```
git branch -M main
```

Ojo. Compruebe `pwd`: debe terminar en `receta-limonada`. Un `init` en la carpeta `home` es un accidente clásico.

Paso 1: crear README.md (sin editor)

Pegue el bloque completo. `cat > crea` (o sobrescribe) el archivo:

```
cat > README.md << 'EOF'
# Receta de limonada
Version de practica para aprender ramas en Git.
Porciones: 2 vasos
EOF
git add README.md
git commit -m "Agrega README de la receta"
```

Compruebe: `cat README.md`. Primer commit: nace el repo. Todavía no está la receta.

Paso 2: crear receta.md (sin editor)

Pegue el bloque completo:

```
cat > receta.md << 'EOF'
# Limonada

## Ingredientes
- Limones: 4
- Agua: 1 litro
- Azucar: 4 cucharadas

## Pasos
- Exprimir los limones.
- Mezclar con el agua y el azucar.
- Servir fria.
EOF
git add receta.md
git commit -m "Agrega receta base de limonada"
git log --oneline
```

Dos fotos en main. Esta es la receta oficial. A partir de aquí nacen las ramas.

Paso 3: variación con menta

cat >> **añade al final**. Dos commits en la rama, no uno. Pegue:

```
git switch -c agregar-menta
cat >> receta.md << 'EOF'

## Variacion
- Menta: 6 hojas
EOF
git add receta.md
git commit -m "Agrega variacion con menta"
cat >> receta.md << 'EOF'
- Machacarlas ligeramente antes de servir.
EOF
git add receta.md
git commit -m "Anota como servir la menta"
git log --oneline --graph --all
```

HEAD está en agregar-menta, con **dos** fotos. main se quedó en la receta *sin* menta.

El momento «ajá»: volver a main

```
git switch main  
cat receta.md  
git log --oneline --graph --all
```

La sección **Variacion no está**. No se borró: vive en la otra rama, en **dos** commits.

Compruebe

En main el archivo termina en los pasos. El **log --all** muestra main en C y la rama verde con **dos** commits (D y E).

Pausa

Estírense, tomen agua, despejen la vista del monitor.

Encargo corto durante el receso

- ① Tengan receta-limonada abierto y **git status** limpio.
- ② Ejecuten **git log --oneline --graph --all**. Deberían ver la rama de menta.
- ③ Si no alcanzaron el init: háganlo ahora, pasos 0 a 3. Al volver fusionamos.

Volvemos para juntar la variación con la receta oficial, un merge limpio y un conflicto de verdad.

Merge = traer el trabajo de una rama a la rama en la que están parados.



- Siempre: **git switch** a la rama **destino**, luego **git merge** la rama **origen**.
- Hoy el destino es **main**: la receta oficial recibe la menta.
- Merge no borra la rama origen. El letrero sigue ahí hasta que lo quiten ustedes.

Dos formas de juntar (hoy alcanzan)

Avance rápido

main no se movió mientras ustedes avanzaban en la variación. Git solo **corre el letrero** de main hasta el último commit de esa rama. No crea foto extra. La rama origen **sigue**.

Commit de fusión

Las dos ramas avanzaron. Git crea una foto nueva con **dos padres**: «aquí se juntaron». El log se ve como un tenedor que se cierra.

Idea clave. No eligen el tipo a mano al empezar. Git mira la historia y decide. Ustedes sí eligen *cuándo* fusionar.

Dibujo: avance rápido

No se crea un commit nuevo. Git solo **mueve el letrero** de main hasta E. D y E son las mismas fotos.



Un commit **no pertenece** a una rama: las ramas son letreros. main en E ve D porque D es su padre.
Si borran agregar-menta, las fotos siguen: solo se fue el letrero verde.

Paso 4: pasar la menta a main

Estén en main. Fusionen la rama de la variación:

```
git switch main
git merge --no-edit agregar-menta
git log --oneline --graph --all
```

```
* a9b8c7d (HEAD -> main, agregar-menta) Anota como servir...
* 3a4b5c6 Agrega variacion con menta
* 1d2e3f4 Agrega receta base de limonada
```

De arriba a abajo: **E**, **D** y **C**. Una columna porque D y E ya salían de C. En E están **los dos** letreros: la rama agregar-menta no se borró.

- **--no-edit** evita el editor del mensaje de merge.
- Si Git dice *Already up to date*, ya habían fusionado: no es un error.

Ojo. El orden importa: primero **switch** al destino (main), después **merge** del origen.

Borrar una rama ya fusionada

Cuando la variación ya está en main, el letrero extra estorba:

```
git branch -d agregar-menta  
git branch  
git log --oneline --graph --all
```

- **-d** (minúscula) borra solo si esa rama **ya** está contenida en la actual. Es el seguro.
- **-D** (mayúscula) borra igual. Hoy **no** lo usamos: puede tirar fotos que no fusionaron.
- Borrar el letrero no borra los commits que ya están en main.

Paso 5: dos variaciones, archivos distintos

Desde main, dos personas tocan **archivos distintos**. Git puede juntarlos sin preguntar.
Pegue:

```
git switch -c cambia-porciones
cat > README.md << 'EOF'
# Receta de limonada
Version de practica para aprender ramas en Git.
Porciones: 4 vasos
EOF
git add README.md
git commit -m "Ajusta la receta a 4 porciones"
```

Paso 5b: hielo (solo receta.md)

Vuelvan a main y abran **otra** rama. cat >> añade al final:

```
git switch main
git switch -c agrega-hielo
cat >> receta.md << 'EOF'

## Servir
- Hielo: 6 cubos
EOF
git add receta.md
git commit -m "Agrega hielo al servir"
git switch main
git merge --no-edit cambia-porciones
git merge --no-edit agrega-hielo
```

Dos merges. Archivos distintos. Status limpio. 4 porciones y hielo.

Cómo se ve un merge limpio en el log

Después de los dos merges:

```
git log --oneline --graph --all
```

Algo parecido a esto (los hashes son de ejemplo, los suyos serán otros):

```
* a1b2c3d Merge branch 'agrega-hielo'
|\
| * e4f5a6b Agrega hielo al servir
* | 7c8d9e0 Ajusta la receta a 4 porciones
|/
* 9c0d1e2 Anota como servir la menta
* 3a4b5c6 Agrega variacion con menta
* 1d2e3f4 Agrega receta base de limonada
```

Idea clave. El dibujo se parte y se vuelve a juntar. Eso no es un error: es dos intenciones que sí caben en la misma receta.

Qué es un conflicto (en la receta)

Un conflicto no es que Git se rompió. Es que **dos fotos tocan el mismo renglón** y Git no adivina cuánta azúcar lleva la limonada.

Variación A

Menos dulce: Azúcar: 2 cucharadas.

Variación B

Más dulce: Azúcar: 6 cucharadas.

- Las dos propuestas son razonables. Git no es el cocinero.
- Por eso deja el archivo a medias y les pide: *elijan ustedes*.
- El choque de hoy es a propósito: la *misma* línea.

Paso 6: menos azúcar

Partan de main. Pegue el bloque entero (cat > reescribe el archivo):

```
git switch main
git switch -c menos-azucar
cat > receta.md << 'EOF'
# Limonada
## Ingredientes
- Limones: 4
- Agua: 1 litro
- Azucar: 2 cucharadas
## Pasos
- Exprimir los limones.
- Mezclar con el agua y el azucar.
- Servir fria.
## Variacion
- Menta: 6 hojas
- Machacarlas ligeramente antes de servir.
## Servir
- Hielo: 6 cubos
EOF
git add receta.md
git commit -m "Reduce el azucar a 2 cucharadas"
```

Paso 6b: más azúcar (otra vez desde main)

La receta oficial **aún** dice 4 cucharadas. Pegue:

```
git switch main
git switch -c mas-azucar
cat > receta.md << 'EOF'
# Limonada
## Ingredientes
- Limones: 4
- Agua: 1 litro
- Azucar: 6 cucharadas
## Pasos
- Exprimir los limones.
- Mezclar con el agua y el azucar.
- Servir fria.
## Variacion
- Menta: 6 hojas
- Machacarlas ligeramente antes de servir.
## Servir
- Hielo: 6 cubos
EOF
git add receta.md
git commit -m "Sube el azucar a 6 cucharadas"
```

Paso 6c: el merge que sí choca

```
git switch main  
git merge --no-edit menos-azucar  
git merge mas-azucar
```

El segundo merge **no** termina. Status dirá *unmerged paths* y `receta.md` marcado como ambos modificados.

No cierren la terminal con pánico

Git está esperando una decisión. Tienen dos salidas honestas: dejar el archivo bueno, o abortar y volver al estado anterior.

Cómo se ve el archivo en conflicto

cat receta.md. Verán algo así (no lo dejen así):

```
<<<<<<< HEAD
- Azucar: 2 cucharadas
=====
- Azucar: 6 cucharadas
>>>>>>> mas-azucar
```

- <<<<<<< HEAD: lo que ya estaba en la rama destino (main, con menos azúcar).
- =====: separa las dos propuestas.
- >>>>>>> mas-azucar: lo que trae la rama que están fusionando.

Ojo. Esos símbolos **no** son decoración. Si quedan en el archivo y confirman, la receta queda rota.

Paso 7: decidir y confirmar (sin editor)

Hoy el cocinero elige **4 cucharadas** (la receta original). Pegue el archivo bueno:

```
cat > receta.md << 'EOF'
# Limonada
## Ingredientes
- Limones: 4
- Agua: 1 litro
- Azucar: 4 cucharadas
## Pasos
- Exprimir los limones.
- Mezclar con el agua y el azucar.
- Servir fria.
## Variacion
- Menta: 6 hojas
- Machacarlas ligeramente antes de servir.
## Servir
- Hielo: 6 cubos
EOF
git add receta.md
git status
git commit -m "Resuelve conflicto de la cantidad de azucar"
```

Si se asustan: abortar el merge

Antes de **add** y **commit** de la resolución, siempre pueden volver atrás:

```
git merge --abort  
git status
```

- Cancela el merge en curso. Vuelven a como estaba main **antes** de ese merge.
- Los commits de menos-azucar y mas-azucar **siguen** en sus ramas.
- Úsenlo si editaron a ciegas y el archivo quedó ilegible. Luego reintentan el merge.

Después de abortar

Pueden volver a **git merge mas-azucar** y resolver con calma. Abortar no es rendirse: es no confirmar un desastre.

- 1 Lean las dos versiones. No borren a ciegas el bloque de abajo.
- 2 Dejen el archivo como debe verse **publicado**: una receta, una cantidad.
- 3 Quiten <<<<<<, ===== y >>>>>> (o reescriban el archivo bueno con **cat >**).
- 4 **git add** del archivo. Status ya no debe decir *unmerged*.
- 5 **git commit** con un mensaje que diga que *resolvieron*, no update.

Idea clave. Resolver un conflicto es una decisión (cuánta azúcar), no un truco de terminal.

Lo que no hacemos hoy (a propósito)

Para no mezclar oficios:

- No usamos **git rebase**. Reescribe historia; sin criterio, asusta más que el conflicto.
- No usamos **git branch -D** ni borramos `main`.
- No hay **stash**, **cherry-pick** ni **reset --hard**.
- No hay **push**: receta-limonada todavía es local. GitHub es la sesión 4.

```
# no ejecute esto en el laboratorio de hoy
# git rebase main
# git branch -D main
```

Si un tutorial de internet pide rebase para «arreglar» el conflicto, cierren la pestaña y pregunten.

Objetivo: el repo receta-limonada con receta oficial, un merge limpio y un conflicto resuelto.

- ➊ Pasos 0 a 2: **init**, `README.md`, `receta.md`, dos commits en `main`.
- ➋ Paso 3: rama `agregar-menta`, **dos** commits, **switch** `main`, **cat** `receta.md`.
- ➌ Paso 4: **merge** de `agregar-menta` en `main`. Opcional: **branch** `-d`.
- ➍ Paso 5: `cambia-porciones` + `agrega-hielo`, dos merges limpios.
- ➎ Pasos 6 y 7: `menos-azucar` y `mas-azucar`, conflicto, **cat** `>` del archivo bueno, commit.

Si ya hicieron los pasos en clase, no los repitan: ejecuten **git log --online --graph --all** y comprueben.

- **Merge en la rama equivocada.** Fusionaron main *dentro* de menos-azucar. Status y rama activa *antes*.
- **Directorio de trabajo sucio al cambiar de rama.** Git se niega. **add+commit** o **restore** *antes* de **switch**.
- **Marcadores de conflicto confirmados.** El archivo aún tiene <<<<<<. Reescriba con **cat >**.
- **git merge abre vim.** Mensaje por defecto: Esc, :wq, Enter. La próxima, **--no-edit**.
- **Pegó sin la línea EOF.** El prompt no vuelve. Escriba EOF y Enter.

Si algo falla: **pwd, ls, git status, git log --oneline --graph --all**.

El dibujo que se llevan a casa

Al terminar, el log no es una línea recta. Eso es el oficio de hoy:

```
* 9f3a21c (HEAD -> main) Resuelve conflicto de la cantidad de azucar
|\
| * 8e2b10d Sube el azucar a 6 cucharadas
* | 7c1a09e Reduce el azucar a 2 cucharadas
|/
* 6b0f08d Merge branch 'agrega-hielo'
```

- Los hashes de la diapositiva **no** son los suyos.
- (HEAD -> main): están en la receta oficial, al día.
- Si el dibujo es una sola línea, falta un merge o están mirando sin **--all**.

Qué no haremos todavía (a propósito)

- No creamos cuenta de GitHub ni hacemos **git push**.
- No clonamos, no hay fork, no hay Pull Request (sesión 5).
- No subimos receta-Timonada a la nube: primero que el historial se entienda en local.
- No usamos rebase ni reescribimos ramas ya fusionadas.

Hoy el oficio es: **partir** el trabajo, **juntarlo** y **decidir** cuando dos fotos no coinciden. Publicar llega cuando sepan *qué* están publicando.

- ① Una rama es un letrero (puntero), no una copia de la carpeta.
- ② `main` es la receta oficial. Las otras ramas son variaciones.
- ③ Tres estados (Pro Git): modificado, preparado, confirmado.
- ④ **`switch -c`** crea y cambia. **`switch main`** vuelve.
- ⑤ **`merge`** junta: primero destino, luego origen.
- ⑥ Conflicto = misma línea, dos intenciones. **`cat`** > del archivo bueno, **`add, commit`**.
- ⑦ **`merge --abort`** es la puerta de emergencia *antes* de confirmar el desastre.

Hoja de comandos de hoy (guárdela)

- ❶ Deje receta-Timonada con **status** limpio y un **log --graph --all** que se parta y se junte.
- ❷ Complete el laboratorio si no alcanzó en clase (pasos 0 a 7, en orden, en esa carpeta).
- ❸ Escriba en un papel, sin mirar: qué es HEAD, qué es merge, qué hace **merge --abort**.
- ❹ Lea Pro Git en español, cap. 3 (ramas y fusiones):
<https://git-scm.com/book/es/v2/Ramificaciones-en-Git-Lo-básico-de-las-ramas-y-fusiones>
- ❺ Traiga una duda escrita sobre *un* conflicto o un merge, no un «no me funciona» suelto.

Próxima sesión: GitHub, **remote**, **push**, **pull** y README en la nube. Esa receta puede subir con ustedes.

- Documentación oficial: <https://git-scm.com/doc>
- *Pro Git* (Chacon y Straub), cap. 1.3 (tres estados) y cap. 3 (ramas):
<https://git-scm.com/book/es/v2>
- **git branch / switch / merge**: páginas git-branch, git-switch, git-merge en
<https://git-scm.com>
- Material de apoyo del curso:
<https://github.com/purpesec/curso-git-github-desde-cero>
- Descarga de Git: <https://git-scm.com/downloads>

Las diapositivas son material original del curso PIT 2026. Los conceptos se contrastaron con Pro Git y con el programa PIT686. El logo de Git tiene ficha en `imagenes/ATRIBUCION.txt`.

Gracias.

Luis Rojas

Sesión 3 · PIT 2026 · Universidad Nacional de Ingeniería

¿Preguntas de último minuto? El chat y el aula virtual siguen abiertos.